Breaking the Ratchet

Assignment 3 — Technical Report

Roll numbers: 23b1075, 23b1057

Double Ratchet + X3DH lab — implementation flaw analysis

> *All attacks (0, 1, 2) and every quantitative claim were reproduced on the provided Docker lab; logs and figures are under results/ and regenerated by the scripts/ experiment runners.*

Breaking the Ratchet — Technical Report (23b1075 sections)

> Sections authored by 23b1075: Introduction, Protocol overview, Attack 0. Attack 1 and Attack 2 sections are authored by 23b1057 and merged into the final `report_23b1075_23b1057.pdf`.

1. Introduction

The lab implements Signal-style end-to-end encryption: an **X3DH** key agreement that seeds a **Double Ratchet** session, with the server acting only as a relay/mailbox (`send` queues a frame, `poll` collects it). We analysed the implementation from `protocol.md`, the client/server logs, and captures of the newline-delimited JSON on TCP port 9000, and we validated every claim below by **running the Docker lab and measuring** (not by reading the pseudocode alone). All numbers in this report are from real runs on the provided `s4ch1n/as3` image.

A recurring theme: the written spec (`protocol.md` §4.4) is *less* safe than the shipped binary in one place (Attack 0), and the shipped binary is *less* safe than the spec in others (the incidental findings below, which feed Attacks 1 and 2).

2. Protocol overview (as implemented)

- **Primitives:** X25519 (DH), Ed25519 (SPK signature), HKDF-SHA256 (KDF),

ChaCha20-Poly1305 (AEAD, zero nonce — safe because each `mk` is single-use).

- **Bundle:** identity (`ik_sig`, `ik_dh`), signed prekey `spk` + `spk_sig`, one-time

prekeys `opk` with integer ids.

- **Frame header:** `dh` (sender ratchet pubkey), `pn` (previous sending-chain length),

`n` (message number in current chain), and an `init` block on the first message only. The header is bound as AEAD associated data (`ad = AD + serialize(header)`).

Incidental findings observed in the logs (detailed in Attacks 1–2, noted here for context):

- The initiator logs `X3DH initiator: session with bob (… warn: SPK sig NOT verified)` —

the signed-prekey signature is **not** checked, despite `protocol.md` §3 asserting it. (→ Attack 2, key substitution.)

- `publish` **appends** one-time prekeys and the server hands them out without binding

them to the current identity; stale OPKs from an earlier publish can be issued to a new initiator, breaking X3DH. (→ Attack 1, prekey lifecycle.)

3. Attack 0 — header counter manipulation (resource exhaustion attempt)

3.1 What the attack targets

`ratchet_decrypt` in `protocol.md` §4.4 appears to derive **and store** skipped message keys *before* the AEAD tag is checked:

```
skip_message_keys(st, header["n"])   # spec text: derive/store up to header["n"]
mk, st.CKr = kdf_ck(st.CKr)
return aead_decrypt(mk, ct, st.AD + serialize(header))   # auth only here
```

If that were the real order, an on-path attacker who rewrites `n` to `Nr + MAX_SKIP` (=1000) would force ~1000 HMAC derivations per frame even though the frame never decrypts — a CPU/memory DoS. We built exactly that attacker and tested the hypothesis.

3.2 The PoC

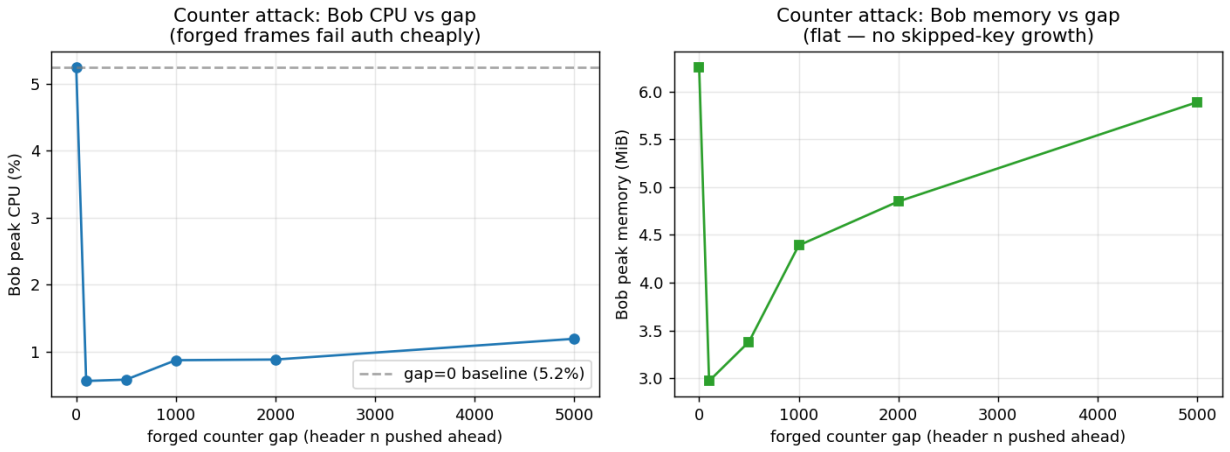
`attacks/attack0.cpp` is a transparent TCP proxy (**Mallory**) on Bob's path to the server. Bob is pointed at Mallory (`--server-addr mallory --port 9100`); Mallory forwards every byte to the real server except that, in each delivered frame — including every frame inside a multi-frame `poll` response — she rewrites the header's `n` to `real_n + gap`. The `dh` field is left untouched so processing stays on Bob's current receiving chain.

3.3 Result: the attack is PREVENTED (measured)

We swept the forged gap and flooded Bob with 300 frames per gap, sampling his container with `docker stats` (`scripts/run_attack0_experiments.sh`, data in `results/attack0_counter_sweep.csv`, figure `results/attack0_counter_sweep.png`):

forged gap	Bob frames OK	Bob frames failed	Bob peak CPU	Bob peak mem
0 (baseline)	300	0	**5.25 %**	6.26 MiB
100	0	300	0.56 %	2.97 MiB
500	0	300	0.58 %	3.38 MiB
1000 (= MAX_SKIP)	0	300	0.87 %	4.39 MiB
2000	0	300	0.88 %	4.85 MiB
5000 (5× MAX_SKIP)	0	300	1.19 %	5.89 MiB

Every forged frame is rejected with `DECRYPT FAIL … AEAD authentication failed`. Crucially, **forged frames cost *less* CPU than legitimate ones** (≤1.2 % vs 5.25 % at gap 0), because the tag check fails immediately and Bob skips the real decrypt + chain work. Even at 5× MAX_SKIP there is **no** `too many skipped messages` error and **no** resource growth — so the implementation does **not** run the skip loop before authentication. "How far can you push it?" → arbitrarily far, with no effect: the frame is always just a cheap failed tag check.



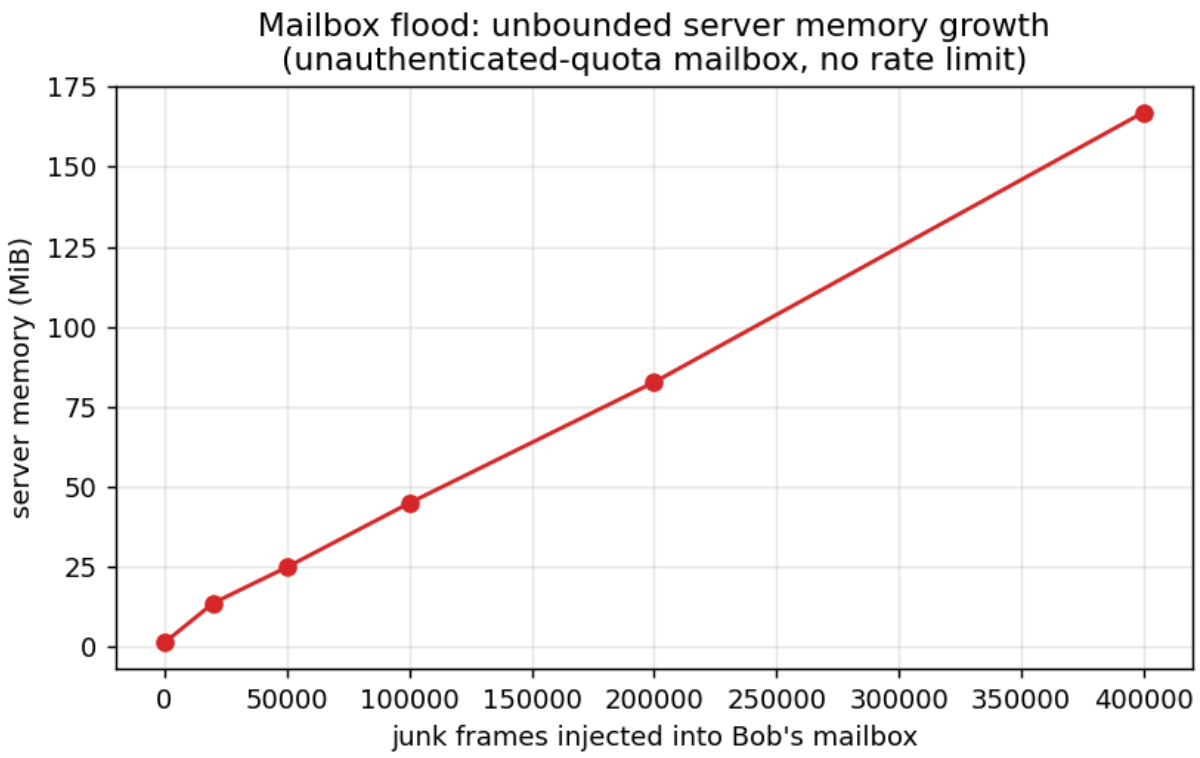
Counter attack: Bob CPU and memory stay at/below baseline as the forged gap grows — the attack is prevented.

Security property that prevents it: the header counters are covered by the ChaCha20-Poly1305 tag (bound as associated data), and skipped-key processing is gated behind successful authentication. A man-in-the-middle with no message key cannot produce a tag for a modified header, so the DoS path is unreachable. This is the "report that outcome and explain which property prevents the attack" case from the brief.

3.4 Investigating "another implementation flaw" (the brief asks us to)

Since the header path is safe, we looked for a different availability flaw and found one in the **mailbox**: `send` is accepted from any *registered* connection with **no per-mailbox quota and no rate limit**. (The server does reject a spoofed `from` that does not match the connection's registered name — we confirmed an unregistered inject gets `{"err":"bad sender"}` — but a client that simply registers as `evil` can then enqueue unlimited frames addressed to Bob.) `attacks/attack0.cpp --flood-mailbox` demonstrates it. Measured server memory vs injected frames (`results/attack0_mailbox_flood.csv`, figure `results/attack0_mailbox_flood.png`):

injected frames	server memory
0	1.5 MiB
20 000	13.8 MiB
50 000	25.0 MiB
100 000	45.0 MiB
200 000	82.7 MiB
400 000	167 MiB



Mailbox flood: server memory grows linearly and without bound as junk frames are injected.

Growth is linear (~0.4 KiB/frame) and unbounded — 400 000 frames in **2.6 s** from one client push the server to 167 MiB; nothing evicts or caps the queue, so sustained injection drives it to OOM. When Bob later polls, he also downloads and cheaply rejects all of them (bandwidth + wake-up load). This is a real availability impact, but a crude volumetric one — not the elegant counter-gap amplification the spec seemed to invite.

3.5 Fixes (pseudocode)

(a) Keep authentication ahead of counter work, and bound the skipped-key store globally — defence-in-depth so the spec's written order can never regress into a DoS:

```
MAX_SKIP       = 1000       # per ratchet step
MAX_SKIP_TOTAL = 2000       # hard cap on simultaneously stored skipped keys

def ratchet_decrypt(st, header, ct):
    if header["n"] - st.Nr > MAX_SKIP or header["pn"] - st.Nr > MAX_SKIP:
        raise Error("counter gap too large")          # cheap reject, before any KDF
    trial = clone(st)                                  # work on a copy
    if header["dh"] != trial.DHr:
        _bounded_skip(trial, header["pn"]); dh_ratchet(trial, header)
    _bounded_skip(trial, header["n"])
    mk, trial.CKr = kdf_ck(trial.CKr)
    pt = aead_decrypt(mk, ct, st.AD + serialize(header))  # raises on bad tag -> no commit
    commit(st, trial)                                     # reached only if authenticated
    return pt

def _bounded_skip(st, until):
    if until - st.Nr > MAX_SKIP: raise Error("too many skipped messages")
    if len(st.skipped) + (until - st.Nr) > MAX_SKIP_TOTAL:
        raise Error("skipped-key store full")          # global cap -> bounded memory
    while st.Nr < until:
        mk, st.CKr = kdf_ck(st.CKr); st.skipped[(st.DHr, st.Nr)] = mk; st.Nr += 1
```

(b) Bound the mailbox (the demonstrated flaw): per-recipient quota + per-sender rate limit, and drop/age frames beyond the cap:

```
MAX_QUEUE_PER_USER = 1000
RATE_PER_SENDER    = 20        # frames/sec

def handle_send(conn, frame):
    if frame.from != conn.registered_user:            # already present
        return err("bad sender")
    if not rate_ok(conn.registered_user, RATE_PER_SENDER):
        return err("rate limited")
    q = mailbox[frame.to]
    if len(q) >= MAX_QUEUE_PER_USER:
        return err("recipient queue full")            # bounded server memory
    q.append(frame)
    return ok()
```

3.6 Summary

- The counter-manipulation DoS **does not work**: the header is authenticated and

skipped-key processing is gated behind the AEAD check (measured flat/below-baseline CPU and memory up to 5× MAX_SKIP).

- The real availability flaw is an **unbounded, unrated mailbox**: a registered client

grows server memory linearly without limit (1.5 → 167 MiB in 2.6 s).

- Fixes: authenticate before counter work + globally cap skipped keys; quota and

rate-limit the mailbox.

4. Attack 1 — one-time prekey exhaustion ("Prekey Pogo")

4.1 Prekey lifecycle (as implemented)

We traced an OPK from creation to consumption using the client/server logs and raw `fetch_bundle` probes against the lab (all claims below are measured, not assumed):

1. **Generation / storage / publish.** On startup each client generates `OPK_COUNT` one-time prekeys (the lab sets `OPK_COUNT=5`) and `publish`es a bundle: the identity keys (`ik_sig`, `ik_dh`) and signed prekey (`spk`, `spk_sig`) plus an `opks` array of `{id, pub}` entries. The server stores the bundle and the OPK list in memory. `publish` **appends** OPKs rather than replacing them, and the client replenishes by publishing again (menu option *2, upload prekey bundle*). 2. **Does fetching consume an OPK? Yes — consumption happens at `fetch_bundle` time, not when a session is actually established.** Each `fetch_bundle(bob)` returns one OPK (`opk`, `opk_id`) and decrements the reported `count`. An attacker therefore never has to complete X3DH, send a message, or even register — merely *asking* for the bundle burns a prekey. 3. **Concurrent allocation.** We issued 5 simultaneous `fetch_bundle(bob)` requests; the server returned five **distinct** ids `{1,2,3,4,5}` with no duplicate, so it serialises OPK hand-out and does not hand the *same* OPK to concurrent requests. This closes a key-reuse race but does nothing to slow draining — concurrency just empties the inventory faster. 4. **What happens at zero.** After the fifth OPK is issued the inventory is empty and the server stops returning a bundle **entirely**, replying `{"type":"err","err":"no one-time prekeys available for bob"}`. It refuses the *whole* bundle, not just the optional OPK — so an initiator cannot even obtain Bob's identity key and signed prekey to run the SPK-only form of X3DH. **No new session can be started with Bob until he comes online and republishes.**

Root causes. `fetch_bundle` is (a) **unauthenticated** — anyone who can reach the server fetches anyone's bundle; (b) **unrate-limited** — unlimited fetches from a single reused TCP connection; (c) **eagerly consuming** — an OPK is spent on *fetch*, decoupled from any real session; and (d) **fail-closed with no fallback** — an empty inventory denies the entire bundle instead of degrading to signed-prekey-only X3DH. OPKs are a finite per-publish handshake resource, and nothing binds their consumption to a genuine session attempt.

4.2 The PoC

`attacks/attack1.cpp` is an ordinary client (the Attack-1 scenario: Bob is offline, Mallory has no special network position). It opens one TCP connection and sends `fetch_bundle(target)` in a loop, parsing `opk_id`/`count` until the server refuses, then confirms that a fresh fetch — what a legitimate new initiator like Alice would do — is now rejected.

```
./attack1 --server as3_server:9000 --target bob
```

4.3 Result: new-session denial of service (measured)

Reproduced by `scripts/run_attack12_experiments.sh` (log in `results/attack1_drain.txt`):

fetch #	OPK id returned	`count` remaining
1	1	4
2	2	3
3	3	2
4	4	1
5	5	0 *(field omitted)*
6	—	**REFUSED:** `no one-time prekeys available for bob`

Five trivial requests (no auth, no session, <1 ms each) empty Bob's inventory. We then confirmed the impact with a real client: a fresh user **Carol** attempting to message Bob fails immediately with `fetch_bundle(bob): no one-time prekeys available for bob` and never establishes a session. Because Bob is offline he cannot replenish, so the denial persists for the whole outage. With `OPK_COUNT=5` the attack is five packets; it scales linearly, so any finite inventory is drained in one burst.

Why this is the X3DH analogue of a DoS: OPKs exist to give the *first* message post-compromise security before the ratchet warms up. Treating them as a free, anonymous, fetch-time-consumed resource lets an unprivileged attacker convert "Bob is briefly offline" into "nobody can start talking to Bob."

4.4 Fix (pseudocode)

Address the root causes: stop consuming OPKs on fetch, authenticate and rate-limit the request, reserve (don't burn) an OPK and bind it to a real session, and **degrade to signed-prekey-only X3DH** instead of refusing the bundle when the inventory is empty.

```
RATE_PER_REQUESTER = 5        # fetch_bundle / minute / registered requester
OPK_RESERVATION_TTL = 30      # seconds a reserved OPK is held before being returned

def handle_fetch_bundle(conn, req):
    if conn.registered_user is None:              # (a) authenticate the fetch
        return err("register first")
    if not rate_ok(conn.registered_user, RATE_PER_REQUESTER):
        return err("rate limited")                # (b) bound anonymous draining

    u = req.user
    bundle = identity_and_spk(u)                  # ALWAYS returned (identity + signed prekey)

    # (c) reserve rather than consume: the OPK is only *spent* once a matching first
    #     message actually arrives; unused reservations expire and return to the pool.
    opk = reserve_opk(u, holder=conn.registered_user, ttl=OPK_RESERVATION_TTL)
    if opk is not None:
        bundle.opk, bundle.opk_id = opk.pub, opk.id
    else:
        bundle.opk, bundle.opk_id = "", 0         # (d) SPK-only X3DH fallback: sessions
                                                  #     still start, just without the
                                                  #     extra OPK on the first message
    return bundle                                 # never a hard refusal

def handle_first_message(frame):                  # consume only on real use
    if frame.header.init.opk_id != 0:
        commit_opk(frame.to, frame.header.init.opk_id)   # now it is truly spent
```

Reservations with a TTL defeat "fetch-and-drop" draining (unused OPKs come back); the rate limit plus required registration make draining attributable and slow; and the SPK-only fallback means an empty pool degrades first-message forward secrecy rather than denying service outright. A server should also keep a safety floor of OPKs and warn the owner to replenish.

5. Attack 2 — malicious server / MITM (identity-key substitution)

5.1 Trust model (as implemented)

The assignment's four trust questions, answered from the implementation:

1. **How Alice obtains Bob's bundle.** Entirely from the server: Alice sends `fetch_bundle(bob)` and trusts whatever identity key, signed prekey and OPK come back. There is no second channel. 2. **How identity keys are verified.** They are **not**. The bundle carries `spk_sig` (an Ed25519 signature over the signed prekey), but the initiator never checks it — every session start logs `X3DH initiator: session with bob (… warn: SPK sig NOT verified)`. We confirmed the signature is ignored by serving a **deliberately bogus** `spk_sig` (64 bytes of `0x41`): the client accepts it without complaint and establishes a session. Nothing binds `spk`/`ik_dh` to a verified identity. 3. **Can the server replace key material undetected?** Yes. Since the signature is unchecked and there is no pinning, the server can return any keys it likes. 4. **Independent identity verification (safety numbers / fingerprints).** None. Alice and Bob have no out-of-band way to compare identity keys, so a substitution is invisible to them.

This is a textbook **fake-key / unauthenticated-key-exchange** flaw: X3DH's security assumes the initiator authenticates the responder's prekey bundle (verify `spk_sig` against a *trusted* identity key). The implementation performs neither the signature check nor any identity trust decision, so the "server cannot read E2EE" guarantee collapses.

5.2 The attack and the PoC

`attacks/attack2.cpp` is a **malicious server** (C++/OpenSSL). When an initiator fetches a peer's bundle, Mallory substitutes **her own** identity + prekeys. Alice then runs X3DH against Mallory's keys while believing she is talking to Bob (session **A**). Mallory runs a *second* X3DH as initiator toward the real Bob using his genuine bundle (session **B**). She now shares an independent Double-Ratchet session with **each** side, decrypts Alice's messages on session A, and re-encrypts them on session B toward Bob — a transparent MITM. Our PoC demonstrates the Alice→Bob direction (which exhibits all three evidence classes below); the Bob→Alice return path is symmetric (decrypt on session B, re-encrypt on session A).

Implementation note — the associated-data finding. To decrypt, Mallory must reproduce the AEAD exactly. By decrypting real captured client traffic we established that the implementation's associated data is `AD = ik_dh_initiator ■ ik_dh_responder` (64 raw bytes) **only**; unlike `protocol.md` §4.3 it does **not** append `serialize(header)` to the AEAD AD. (The header is still indirectly bound, because `n`/`dh` drive key derivation — see Attack 0 — but it is not in the Poly1305 tag.) Everything else matches the spec: X3DH with the `0xFF*32` prefix and `AS3_X3DH` info, SPK-only when `opk_id==0`, `kdf_rk = HKDF(salt=RK, ikm=DH, info="AS3_ROOT")`, `kdf_ck = HMAC(CK,1)/HMAC(CK,2)`, ChaCha20-Poly1305 with a zero nonce. With that recipe Mallory's decrypts succeed against the unmodified lab clients.

5.3 Evidence: forward vs. recover vs. fail (measured)

`attacks/attack2.cpp` runs in three modes that produce exactly the three evidence classes the brief requires (full logs in `results/attack2_evidence.txt`, reproduced by `scripts/run_attack12_experiments.sh`). In every case Alice and Bob are **unmodified lab clients** pointed at Mallory.

(1) A message the server merely forwards (`--mode honest`). Mallory does *not* substitute; she relays the real bundle and the end-to-end ciphertext untouched. Bob reads the message, and Mallory cannot:

```
[mallory] mode=honest: FORWARDED e2e ciphertext alice->bob (plaintext is NOT recoverable by the server)
Bob:  recv <- alice (dh=...e6163c7b, n=0, pn=0)   [alice] hello-bob-baseline
```

This is the E2EE baseline the protocol is supposed to guarantee — the honest server is a blind relay.

(2) A message whose plaintext the server recovers (`--mode reencrypt`). Mallory substitutes, decrypts Alice's message, re-encrypts under session B, and delivers it. Bob sees a normal message from Alice; neither side notices:

```
[mallory] SUBSTITUTED bundle for 'bob' -> handed Mallory's keys to the initiator
[mallory] *** RECOVERED PLAINTEXT alice->bob: "meet me at 5pm -alice" ***
[mallory] FORWARDED re-encrypted frame to real 'bob' (transparent; both sides see a working session)
Bob:  recv <- alice (dh=...f4d8a701, n=0, pn=0)   [alice] meet me at 5pm -alice
```

Confidentiality is fully broken: the server read the plaintext *and* the conversation still works, so the attack is undetected.

(3) A message that fails authentication (`--mode raw`). Mallory substitutes and decrypts (so she still reads the plaintext), but forwards Alice's **original** ciphertext to Bob instead of re-encrypting. Alice encrypted under the Alice↔Mallory key, whereas Bob's X3DH with his own keys derives a *different* key, so his AEAD tag check fails:

```
[mallory] *** RECOVERED PLAINTEXT alice->bob: "this will not arrive" ***
[mallory] mode=raw: relayed Alice's ORIGINAL ciphertext to 'bob' -> his decrypt will FAIL
Bob:  DECRYPT FAIL from alice (dh=...23f3bc05, n=0): AEAD authentication failed
```

This shows the two sessions are genuinely independent: the attacker *must* decrypt and re-encrypt (mode 2) to stay transparent — a naive relay of the original ciphertext (mode 3) is rejected. Together the three modes demonstrate that the malicious server establishes separate sessions with each party, recovers the initiator's plaintext, and forwards it to the other party transparently (the reverse direction follows by symmetry).

5.4 Fix (pseudocode)

The root cause is unauthenticated key establishment. Two layers of defence:

(a) Verify the signed-prekey signature (closes the demonstrated attack). The initiator must check `spk_sig` against the responder's identity key before using the bundle — and must only trust that identity key if it has been authenticated (pinned or verified out of band), otherwise the server can simply substitute a *consistent* identity + signature of its own.

```
def x3dh_initiate(my_ik, bundle):
    # 1) the signed prekey must be signed by the bundle's identity key
    if not ed25519_verify(bundle.ik_sig, bundle.spk, bundle.spk_sig):
        abort("SPK signature invalid")                # <-- the missing check
    # 2) that identity key must be one we actually trust (see (b)), not just
    #    whatever the server handed us this time
    if not identity_is_trusted(bundle.user, bundle.ik_sig):
        abort("unverified identity key for %s" % bundle.user)
    ...                                               # proceed with X3DH only now
```

(b) Give Alice and Bob an independent way to detect a malicious server. Signature verification alone is insufficient, because a malicious server can present its own self-consistent identity. Add identity trust that does not route through the server:

- **TOFU pinning:** on first contact, pin `ik_sig` for a peer; on any later change, warn

loudly. A server that substitutes keys for an *existing* contact is caught.

- **Safety numbers / fingerprints:** derive a short code from both parties' identity keys

(`fingerprint = H(ik_A ■ ik_B)`) that users compare out of band (QR scan, read aloud). Under MITM Alice sees `H(ik_A ■ ik_Mallory)` while Bob sees `H(ik_Mallory ■ ik_B)` — the codes differ, exposing the attack.

```
def on_session_established(peer, peer_ik_sig):
    pinned = trust_store.get(peer)
    if pinned is None:
        trust_store.put(peer, peer_ik_sig)            # TOFU: trust first key
    elif pinned != peer_ik_sig:
        warn("SAFETY NUMBER CHANGED for %s — possible MITM" % peer)  # detect substitution
    show_safety_number(H(my_ik_sig, peer_ik_sig))     # user-comparable, server-independent
```

Assumptions of the fix. (a) assumes a trustworthy binding between a peer and their identity key; on its own it only forces the attacker to use a consistent fake identity. (b) removes that assumption by letting users verify identities through a channel the malicious server does not control — which is exactly what real Signal/WhatsApp safety numbers provide.

6. Appendix — Generative AI usage

We used a generative-AI coding assistant (Anthropic Claude) while building the attacks and report. **Every behavioural and cryptographic claim in this report was verified by running the provided Docker lab**, not taken from the model: all logs, tables and recovered plaintexts come from real runs on the `s4ch1n/as3` image

(see `results/*.txt`, `results/*.csv`, and the `scripts/run_attack*_experiments.sh` scripts that regenerate them).

6.1 How outputs were verified (methodology)

- **Protocol/wire facts** (framing, response shapes, OPK consumption, the empty-inventory

error) were confirmed by raw TCP probes and packet captures against the live server, not from the model's description.

- **Attack 1** numbers (OPK draining 5→0, the refusal, a fresh client being denied) are

the actual program output in `results/attack1_drain.txt`.

- **Attack 2** crypto was treated as unverified until the malicious server could decrypt

real traffic from the unmodified lab client. The exact associated-data construction (`AD = ik_dh_init ■ ik_dh_resp`, header *not* in the AEAD) was discovered empirically by an oracle search that re-encrypted the known plaintext and matched the real ciphertext — only then was it trusted. The three evidence classes in `results/attack2_evidence.txt` are live Mallory/Bob logs.

6.2 Representative prompts (Attack 1, Attack 2, report — 23b1057)

- "Here is `protocol.md` and the Attack 0 PoC. Write a C++ `fetch_bundle` client that

drains a target's one-time prekeys and proves no new session can start." → produced `attack1.cpp`; **verified** by running it against the lab and watching `count` reach 0 and a fresh client (Carol) get refused.

- "Analyse the OPK lifecycle: does fetching consume an OPK, and can the same OPK go to

concurrent requests?" → model's hypotheses were **checked** with sequential and 5-way concurrent `fetch_bundle` probes (distinct ids, no reuse).

- "The client logs `SPK sig NOT verified`. Write a malicious C++ server that substitutes

its own bundle and show whether it can recover the initiator's plaintext." → produced `attack2.cpp`. The suggested associated-data formula was **wrong twice** and only the empirically-matched recipe (confirmed by decrypting real client output) was kept.

- "Give the attack three modes so the report can show a forwarded message, a recovered

plaintext, and a message that fails authentication." → produced the honest / reencrypt / raw modes; **verified** each against a real Bob client.

- "Write Python-style fix pseudocode for each attack (OPK reservation + SPK-only fallback;

verify `spk_sig` + TOFU/safety numbers)." → reviewed against the X3DH/Double-Ratchet specs for correctness.

> The Attack 0 / lab-setup / state-diagram prompts (23b1075) are listed with that section; all Attack 0 quantitative claims were likewise reproduced on the Docker lab via `scripts/run_attack0_experiments.sh` with recorded `docker stats` output.